ETL Example—Scheduling

Understand how to automate ETL pipeline execution using the cron scheduler on Unix systems. This lesson guides you through creating cronjobs to run ETL scripts every Monday at 9 AM, ensuring timely data refreshes without manual intervention.

The company wants us to schedule the ETL pipeline we built so it would run once a week on Monday at 09:00 a.m. That way, the data scientist will have updated data as new data about lottery numbers comes in without bothering us to deploy the ETL processes repeatedly. To schedule the ETL pipeline, we'll use **cron**.

Cron

Cron is a command-line utility for scheduling jobs on Unix operating systems. Assuming we're operating on one, we can easily schedule commands or shell scripts to run automatically and on a schedule. Tasks scheduled using cron are also known as **cronjobs**.

Cron is a very useful tool for repetitive tasks like the one we just built. To create cronjobs, we must insert the right syntax in the **crontab**, a file that stores all the scheduled jobs.

We can edit the crontab file and add tasks by using the command `crontab -e`. This will launch an interface provided by the system for users to manipulate their cronjobs and store them in `/var/spool/cron/crontabs`.

Ask

Syntax

The syntax for creating a cronjob is five characters representing the time interval, followed by the command to run.

index.md Markdown

```
Syntax:
┌──────────────── minute
│ ┌──────────────── hour
│ │ ┌──────────────── day of the month
│ │ │ ┌──────────────── month
│ │ │ │ ┌──────────────── day of the week
│ │ │ │ │
│ │ │ │ │
│ │ │ │ │
* * * * * <Command To Execute>
```

Cronjob syntax: Five characters followed by a Bash command

Each of these asterisks (*) represents a time interval for the jobs. From the left are minutes, hours, day of the month, month, and day of the week. Here are the accepted values for each field:

index.md Markdown

```
Field              |Accepted Values
----------------- |--------------
minute             |0-59
hour               |0-23
day of the month   |1-31
month              |1-12 or names (see below)
day of the week    |0-7 (Sunday is 0 or 7)
*                      |any value
,                      |value list separator
-                      |range of values
/                      |step values
```

Ask

Accepted values for each character in the cronjob

Cron will execute jobs when the current time matches the minute, hour, and month of the cronjob and either one of the “day of the month” or “day of the week.”

Cronjob examples

- `* * * * * uptime >> uptime_logs.txt`
  - Every minute, it runs the command `uptime` and appends the output to the `uptime_logs.txt` file.
- `0 0 * * * free -m`
  - Every day at 00:00, it runs the command `free -m` to check the available memory in the system.
- `@weekly cd /root/ETL_Project && bash run_etl.sh`
  - Every week at 00:00 on Sunday, it changes directories to `/root/ETL_Project` and runs an ETL process using the `bash run_etl.sh` files.

Using cron

Let's schedule the ETL pipeline to run every Monday at 09:00 AM. In this case, the schedule interval will be `0 9 * * 1`, which translates to “Every Monday at 09:00,” and the command will be `cd /PathToFile && bash main.sh`, which will change directories to the location of `main.sh` and run the file.

The final cronjob should look like this: `0 9 * * 1 cd /usercode/ETL_Lot` `&& bash main.sh`.

Ask

Go over the following steps in the terminal widget below to schedule the ETL pipeline:

1. To use cron, the first thing we must do is start the cron service using the `service cron start` command. We should see an output message: `"Starting periodic command schedular: cron"`.
2. Next, we need to insert a cronjob into the crontab file. We can do that by running the `crontab -e` command, which will open the system's built-in user interface. The system will ask us which text editor we want to edit the crontab with. I recommend choosing “nano” because it’s easier and more straightforward. To choose nano, press 1.
3. Finally, to schedule the ETL pipeline, use the arrows (↑ ↓ → ←) to scroll to the end of the file and insert a schedule and a command according to the company’s needs.
4. To save the file, use “CTRL + O” followed by the “Enter” key.
5. To exit the file, use the “CTRL + X” key combination.

Terminal 1

Terminal

Loading...

Ask

After finishing editing the crontab, the file should look like this:

main.sh Shell

```shell
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
```

Scheduling the ETL using the “crontab -e” command

The final result is that every Monday at 09:00 AM, our Bash ETL pipeline will run, and the data scientist will have a fresh batch of lottery data to match their analytic needs.

Ask